

Welcome to D7065E, Embedded Intelligence at the Edge

This course teaches how to design and build **autonomous intelligent systems** that sense, reason, and act in physical environments. The focus is on the intersection of edge computing, AI agents, and cyber-physical systems (CPS).

Course syllabus:

<https://www.ltu.se/en/education/syllabuses/course-syllabus?id=D7065E>

You will learn to:

- Design cyber-physical system (CPS) architectures that connect sensors, computation, and actuators into autonomous control loops.
- Apply model-based and specification-driven engineering methods to design, implement, and validate cyber-physical systems.
- Build data pipelines that collect, store, and transform sensor data for monitoring, analytics, and decision-making.
- Develop autonomous services and agents that observe system state, reason about conditions, and take action through software interfaces.
- Evaluate the resilience, safety, scalability, and performance of autonomous systems under failures, uncertainty, and changing operating conditions.



The course project centers on autonomous building control: you instrument a simulated smart building with sensors and actuators, then develop software that makes real-time control decisions. The project is intentionally open-ended: students may choose different control problems, architectures, communication patterns, and autonomous approaches. There is no single correct solution.

The course project will be built upon using a smart building simulator of the LTU A-house. More information can be found here: <https://github.com/eislab-cps/D7065E>

[Länkar till en extern sida.](#)

Expectations

- This is a systems engineering course. The primary focus is on architecture, communication, data pipelines, testing, and system integration.
- The AI component is intended to demonstrate autonomous decision-making, not advanced AI research. Rule-based agents, machine learning models, and LLM-based solutions are all acceptable.
- You are expected to apply fundamental distributed systems concepts, including service decomposition, inter-process communication, APIs, containerization, and fault handling.
- You are **not** expected to implement advanced distributed systems algorithms such as consensus protocols, distributed databases, or large-scale cloud infrastructure. These topics are covered in later courses.
- Every design choice should be justified in terms of system requirements, trade-offs, and operational considerations.
- Both team members must understand and be able to explain the complete system architecture and its components.

AI-Assisted Development

The use of modern **agentic AI development tools** is strongly encouraged in this course. Tools such as Claude Code, OpenAI Codex, Cursor, Windsurf, Cline, and Aider can autonomously read codebases, edit files, run commands, write tests, and assist with system design and implementation. Agentic coding tools are becoming a major part of professional software engineering workflows and can dramatically increase development speed and project scope. However, you remain responsible for all design decisions, code, tests, and conclusions. The oral examination assesses your understanding of the system, not the AI's ability to generate it.

Examination

The project may be completed individually or in pairs. At the end of the course, each student will participate in an individual oral examination. The examination focuses on architectural understanding, design decisions, and demonstration of the project implementation.

A pair submits one system, repository, report, and joint demonstration. For pair projects, the report must contain a contribution statement describing each student's concrete contributions. The oral examination is individual. Each student must understand and be able to explain the complete system, not only their own contribution. Students in the same pair may receive different grades.

Specification-First Development

Most developers write code first, then tests, then discover the design was wrong. With AI coding tools, this problem can become even worse, as the AI may generate clean-looking code that satisfies the prompt rather than the underlying requirements.

This course uses **Model-Based Systems Engineering (MBSE)**

[Länkar till en extern sida.](#)

: you design the system with precise models (component diagrams, sequence diagrams, data flows, requirements) before writing any code. These models become the input to AI coding tools. Tests are derived from the specification, not invented after the fact.

Other reading material

Architecture and Systems Engineering

- **The C4 Model** — Simon Brown
 - Lightweight architecture documentation used throughout the course.
 - <https://c4model.com/>
- **Martin Fowler — Microservices Resource Guide**
 - Practical discussion of service decomposition, APIs, and system boundaries.
 - <https://martinfowler.com/microservices/>
- **Google Site Reliability Engineering (selected chapters)**

- Reliability, monitoring, failure handling, and operational thinking.
 - <https://sre.google/books/>
-

Distributed Systems

- **Martin Kleppmann — Distributed Systems (Cambridge)**
 - Required: Lecture 1.1 Introduction to Distributed Systems.
 - Recommended: additional lectures as needed.
 - **Designing Data-Intensive Applications — Martin Kleppmann**
 - Especially chapters on data models, messaging, streams, and system design.
 - Often considered the modern introduction to distributed systems.
-

Cyber-Physical Systems and Digital Twins

- **System-Level Simulation (Wikipedia)**
 - Useful introduction to simulation-based engineering and CPS validation.
 - https://en.wikipedia.org/wiki/System-level_simulation
- **Digital Twin (Wikipedia)**
 - Background on simulation-driven

engineering and virtual system validation.

- https://en.wikipedia.org/wiki/Digital_twin

Data Engineering

- **TimescaleDB Documentation**
 - Time-series data modelling and storage.
 - <https://docs.timescale.com/>
- **Apache Kafka: Event Streaming 101**
 - Good overview of event-driven architectures and data pipelines.
 - <https://developer.confluent.io/>

(Not required, but useful for students interested in data-centric architectures.)

Autonomous Systems and AI Agents

- **Andrej Karpathy — Intro to Large Language Models**
 - Required background on modern LLMs.
- **Harrison Chase — State of AI Agents**
 - Practical overview of agent architectures and tooling.
- **Anthropic — Building Effective Agents**
 - One of the best concise introductions to modern agent design patterns.

- <https://www.anthropic.com/engineering/building-effective-agents>
 - **ReAct: Synergizing Reasoning and Acting in Language Models**
 - Classic paper introducing the ReAct pattern used in many agent frameworks.
-

Testing and Reliability

- **Google Testing Blog**
 - Practical testing strategies for complex systems.
 - <https://testing.googleblog.com/>
- **Release It! — Michael Nygard**
 - Excellent discussions of failure modes, resilience, cascading failures, and fault isolation.